

NetGuard AI: A Lightweight Machine Learning-Based Dashboard for Network Anomaly Detection and Risk Monitoring

Achmad Maulana, 241730016

Department of Informatics, Faculty of Science and Technology
Universitas Islam Negeri Sultan Maulana Hasanuddin Banten
Banten, Indonesia

e-mail : 241730016.achmadmaulana@uinbanten.ac.id

Abstract: Network anomaly detection is needed by educational institutions that operate computer laboratories, local servers, and internet access with limited monitoring resources. This paper presents NetGuard AI, a low-cost machine learning system for distinguishing normal and anomalous traffic and presenting risk levels through a lightweight web dashboard. The study uses a representative subset of CICIDS2017 Friday-WorkingHours-Afternoon-DDos. The preprocessing stage includes column-name normalization, binary label conversion, missing and infinity value handling, numerical feature selection, duplicate removal, and processed-data export for reproducible experiments. Logistic Regression, Decision Tree, and Random Forest are compared using accuracy, precision, recall, F1-score, and confusion matrix. The best model is selected using F1-score as the primary criterion, recall as the secondary criterion, and accuracy as the tertiary criterion. From 19,935 processed records with an 80:20 data split and `random_state=42`, Decision Tree achieved accuracy 0.9992, precision 0.9995, recall 0.9990, and F1-score 0.9992. The contribution of this study is the integration of anomaly detection, automatic risk classification, recommended actions, and a reproducible artifact package that can run on a standard laptop without paid APIs.

Keywords: network anomaly detection, CICIDS2017, machine learning, intrusion detection system, risk dashboard

I. INTRODUCTION

The growth of digital services in schools, universities, and small organizations has made network monitoring increasingly important. Administrators need to know whether incoming traffic remains normal or has started to indicate an attack pattern. In many educational environments, this need is not always supported by commercial security information and event management tools, dedicated sensors, or a full-time security operations team.

NetGuard AI is developed to address this need through an approach that students can reproduce and study. The system connects computer networking concepts with supervised machine learning, from traffic data, preprocessing, model training, and evaluation to risk interpretation. The research question is how a low-cost machine learning pipeline can detect anomalous traffic and present the result in a form that is useful for beginner administrators.

This article follows the IEEE structure consisting of title, abstract, keywords, introduction, related works, proposed method, experimental setup, results and discussion, conclusion, and references. This structure is selected so the contribution is not merely a working program, but also an academic artifact that presents the problem, literature position, method, experimental evidence, and limitations.

The novelty of this study is not placed in the creation of a new algorithm. The main contribution is the packaging of a CICIDS2017-based anomaly detection experiment into an educational workflow: clear baselines, prediction results translated into a risk score, understandable recommended actions, and artifact documentation that allows an examiner to trace the data, code, model, results, and dashboard.

The practical motivation of this study comes from the fact that many network monitoring tasks in small environments are still reactive. Administrators often inspect logs only after a connection slows down, users complain, or a device becomes unreachable. This workflow may be too late when volumetric attacks or traffic spikes occur quickly. A simple classification model does not replace administrator expertise, but it can provide an early signal for prioritizing inspection.

In an educational context, systems that are too complex are often difficult to explain in class. NetGuard AI intentionally keeps the models familiar, the metrics standard, and the dashboard simple. This choice makes the relationship between data, model, and decision easier to observe. Students can see that accuracy, precision, recall, and F1-score are not merely numbers, but have implications for false alarms, missed attacks, and monitoring workload.

The scope is also clearly limited from the beginning. This study does not claim to be a production intrusion detection system that can replace enterprise security devices. Its focus is to build a reproducible proof of concept that can be retested. With this position, the evaluation emphasizes pipeline consistency, dataset suitability, metric clarity, and the dashboard's ability to connect experimental results with beginner user needs.

II. RELATED WORKS

Public datasets such as CICIDS2017, CICIDS2018, UNSW-NB15, and NSL-KDD are widely used in intrusion detection research because they provide labeled traffic and network-flow features that can be processed by machine learning algorithms [1], [6], [7], [20]. CICIDS2017 is practically useful because it provides benign traffic and several common attacks in PCAP and CSV flow formats. However, several studies also show that benchmark datasets must be used carefully because duplicates, labeling errors, and improper data splits may produce overly optimistic results [3]-[5].

The literature indicates that classical models such as Logistic Regression, Decision Tree, and Random Forest remain relevant for tabular network features, especially when the research objective requires explainable and reproducible baselines [2], [10], [13], [14], [16]. Tree-based models are often strong for non-linear patterns, while linear models provide a simple comparison point. Recent studies also show that hybrid and deep learning approaches can improve performance in some scenarios, but their complexity is not always aligned with low-cost educational needs [8], [9], [11].

From this literature, three gaps are relevant to this study. First, many studies focus on metric values without providing an interface that helps readers understand the operational impact of prediction results. Second, research often presents complex models, whereas educational environments need solutions that can run on ordinary laptops. Third, classification metrics are rarely translated into risk levels and recommended actions that beginner administrators can understand.

NetGuard AI addresses these gaps by combining classification experiments, visualization, risk scoring, and a reproducibility package. This position remains realistic because the study does not claim to outperform all modern IDS approaches. The emphasis is on pipeline transparency, artifact traceability, and educational

usefulness. Therefore, numerical results are interpreted together with dataset validity, computational cost, and the need to communicate results to non-specialist users.

Studies on CICIDS2017 also show the importance of methodological caution. The dataset is popular because it provides modern traffic, attack labels, and flow features ready for processing, but popularity does not automatically make every experiment valid. Common errors may occur when duplicate records are not cleaned, when the label column is mixed with features, or when train-test splitting causes data leakage. For this reason, preprocessing in this study is treated as a scientific stage that is as important as model training.

Previous work on Random Forest and tree-based models explains why this model family often performs well on tabular data [13], [14]. A decision tree can form local rules based on feature thresholds, while Random Forest reduces dependence on a single tree structure. However, a stronger model is not always the most appropriate for every objective. In a learning project, interpretability, training speed, and ease of demonstration are also important considerations.

Classification evaluation literature emphasizes that metric selection should follow the consequence of errors [17]-[19]. In anomaly detection, false negatives are related to malicious traffic that is not detected, while false positives are related to additional alerts that must be inspected. Therefore, this study does not use accuracy as the only measure. F1-score is selected to maintain balance, while recall remains important because missed attacks have more serious security consequences.

The connection between research and implementation artifacts is an important distinction. Many papers present final results in tables, but readers cannot always trace how those results were produced. NetGuard AI provides metric files, model comparison, prediction results, visualizations, and a dashboard so that the claims in the article can be checked through project outputs. This approach is consistent with reproducibility principles in machine learning research [16].

TABLE I. LITERATURE POSITIONING SUMMARY

Literature Focus	Main Finding	Relevance to NetGuard AI
CICIDS2017 and dataset quality	The dataset is useful but requires careful preprocessing and data splitting	The pipeline is transparent and processed data is stored
Classical IDS models	Logistic Regression, Decision Tree, and Random Forest remain strong for tabular features	Models are compared as auditable baselines
Deep learning approaches	Performance can improve, but computational cost and complexity are higher	The solution targets ordinary laptops and learning
Educational deployment	Many studies do not connect metrics with user action	The dashboard translates anomalies into risk and recommendations

III. PROPOSED METHOD

The proposed method consists of dataset acquisition, preprocessing, model training, evaluation, prediction export, dashboard visualization, and artifact packaging. Raw data is stored separately from processed data so the experiment can be audited. Preprocessing normalizes column names, identifies the label column, converts BENIGN into class 0, converts DDos into class 1, keeps numerical features, replaces infinity values, fills missing values using the median, and removes duplicate records.

The training stage compares Logistic Regression, Decision Tree, and Random Forest. Logistic Regression is used as a fast and explainable linear baseline. Decision Tree is used as a non-linear baseline that can capture feature-splitting rules without heavy

computation. Random Forest is used as an ensemble comparator because it combines many decision trees and is usually more stable against data variation [13].

Model selection uses F1-score as the primary criterion, recall as the secondary criterion, and accuracy as the tertiary criterion. This order is used because anomaly detection should not optimize only general correctness. Precision helps reduce false alarms, recall helps reduce missed attacks, and F1-score balances both measurements [17]-[19].

The dashboard reads metrics.json, model_comparison.csv, prediction_result.csv, and research_summary.json. The risk score is computed from the anomaly ratio. Low risk is assigned when the anomaly ratio is below 10 percent. Medium risk is assigned when the ratio is from 10 percent to below 30 percent. High risk is assigned when the ratio reaches 30 percent or higher. This risk classification helps convert model output into a more readable operational message.

The method is designed so every stage has a clear input and output. Raw data becomes the input of preprocessing. Processed data becomes the input of training. The best model and prediction results become the input of evaluation and dashboard visualization. This separation makes errors easier to trace because changes in one stage are not directly mixed with other stages. The design also helps examiners verify whether the numbers in the article come from the same experiment output files.

Logistic Regression is retained even though it is not the best model because a linear baseline gives context to dataset difficulty. If a linear model already performs well, some features likely provide a clear separation boundary. If a tree-based model improves far beyond the linear baseline, the improvement can be linked to its ability to capture non-linear patterns. Therefore, the baseline is not a formality, but a tool for reading the character of the data.

Decision Tree is selected as a comparator because it has educational value. The tree structure can be explained through thresholds, branches, and decision leaves. In a classroom discussion, this model helps explain why a record is assigned to the normal or anomalous class. Although a single tree can be vulnerable to overfitting, it remains useful as a comparator because it is computationally light and directly interpretable.

Random Forest is used to observe whether combining many trees provides additional stability. In theory, an ensemble reduces the variance of a single model and often performs strongly on tabular data [13]. In this study, Random Forest is not forced to become the final model. The best model is selected using a predefined rule, so the decision does not depend on subjective preference for a particular algorithm.

The dashboard component does not only function as a final display, but also as a risk communication mechanism. The risk score simplifies the distribution of predictions into a value that can be read quickly. Recommended actions then connect the risk level with practical steps, such as checking logs, limiting access temporarily, or conducting further investigation. In an educational context, this function helps readers understand why classification output should be translated into operational language.

IV. EXPERIMENTAL SETUP

The experiment uses a representative subset of CICIDS2017 Friday-WorkingHours-Afternoon-DDos. After preprocessing, the data contains 19,935 records, consisting of 9,935 normal records and 10,000 anomalous records. To maintain reproducibility, the data is split using an 80:20 train-test split with random_state=42. This configuration is selected so the process can be rerun on a standard laptop without a GPU.

The evaluation metrics are accuracy, precision, recall, F1-score, and confusion matrix. Accuracy describes overall correctness. Precision

indicates the proportion of predicted anomalies that are truly anomalous. Recall indicates the proportion of anomalous traffic successfully detected. F1-score is used as the main measure because it combines precision and recall into a more balanced value for binary classification.

The experiment is intentionally low-cost. The core experiment uses Python and open-source libraries, so it does not require a paid API, database server, or commercial SIEM device. This design decision aligns with the purpose of the study, which is to produce an artifact that students can inspect, rerun, and extend in a network security learning context.

The 80:20 split is used to balance training and testing needs. The training portion is used to build the model, while the testing portion is used to measure performance on data unseen during training. The use of `random_state=42` makes the split repeatable. Without this setting, the result could change each time the experiment is run, making the article discussion harder to verify.

The processed dataset remains relatively balanced between normal and anomalous traffic. This condition helps metric interpretation because accuracy is not strongly dominated by one class. Nevertheless, the study still reports precision and recall because real network environments are not always balanced. In operational implementation, normal traffic is usually far more frequent than attack traffic, so thresholds and alarm policies must be recalibrated.

The experiment configuration also considers hardware limitations. The selected models can be trained without a GPU and do not require a special server configuration. This criterion is important to ensure that the artifact works not only on the project creator's computer, but can also be tested by other lecturers or students. This kind of reproduction is part of research quality, especially for academic assignments that require direct inspection.

Experiment outputs are stored in JSON, CSV, and image formats so they can be used by multiple components. JSON files contain key metrics and dataset summaries. CSV files store model comparisons and prediction results. Images present class distribution, model comparison, confusion matrix, and risk score. Open formats make results easier to read, inspect, and reuse in the dashboard.

TABLE II. EXPERIMENTAL SETUP

Component	Configuration
Dataset	CICIDS2017 Friday-WorkingHours-Afternoon-DDos
Processed data	19,935 records
Data split	80 percent training and 20 percent testing
Models	Logistic Regression, Decision Tree, Random Forest
Selection rule	F1-score, recall, accuracy
Output	Metrics, confusion matrix, prediction result, dashboard

Distribusi Dataset CICIDS2017 Subset

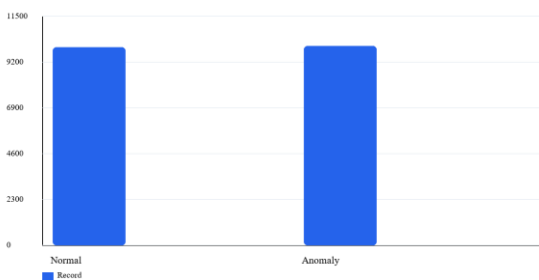


Fig. 1. Class distribution of the representative CICIDS2017 subset.

Perbandingan Model NetGuard AI

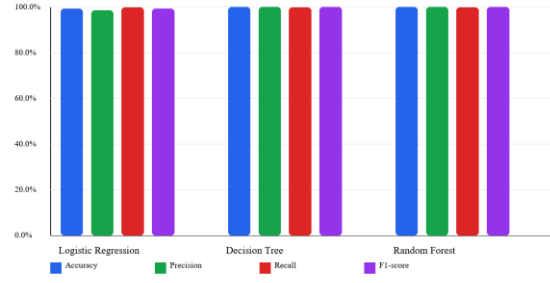


Fig. 2. Performance comparison across Logistic Regression, Decision Tree, and Random Forest.

V. RESULTS AND DISCUSSION

The experimental results show that Decision Tree is the best model with accuracy 0.9992, precision 0.9995, recall 0.9990, and F1-score 0.9992. Random Forest achieved perfect precision on this split, but its F1-score is slightly lower than that of Decision Tree. Logistic Regression also shows strong performance, indicating that the DDoS flow features in this subset have a clear separation pattern after preprocessing.

The confusion matrix shows a very low number of errors: one false positive and two false negatives for the Decision Tree model. This result supports the effectiveness of the model on the tested subset, but it must not be interpreted as universal deployment readiness. Real network traffic may drift, attack types may be mixed, and normal campus traffic may be more diverse than the benchmark subset.

From an operational interpretation perspective, recall is important because a false negative means anomalous traffic is predicted as normal and may be missed by the administrator. Precision is also important because a false positive can increase investigation workload. Therefore, the discussion does not stop at accuracy, but connects precision, recall, F1-score, confusion matrix, risk score, and recommended actions.

The dashboard strengthens the practical contribution of the study. The interface displays total records, normal traffic count, anomaly count, best-model accuracy, risk score, risk level, recommended actions, model comparison chart, and confusion matrix. The prediction page allows users to upload a CSV file and obtain a classification summary. This form makes experimental results easier for beginner administrators to understand than raw metric tables.

The main threat to validity is dataset coverage. The experiment is still limited to one CICIDS2017 DDoS subset, so it does not represent all real network conditions. Other threats include benchmark bias, simple hyperparameter tuning, and a dashboard that remains a prototype. Further validation should include multiple attack types, other datasets, broader cross-validation, and campus traffic collected under ethical permission.

The model comparison shows that all algorithms achieved high performance, but the results must still be read carefully. Logistic Regression achieved accuracy 0.9920 and F1-score 0.9921. This value is already strong for a linear baseline. Decision Tree and Random Forest both achieved accuracy 0.9992, but Decision Tree is slightly better in F1-score. A small difference like this should be read together with the confusion matrix, not only through a single summary number.

For the Decision Tree model, the false positive count is one record. This error means one normal traffic record is marked as anomalous. In monitoring practice, this may cause additional inspection. The workload is usually acceptable if the alert count is not too large. The false negative count is two records and is more sensitive from a

security perspective because anomalous traffic is not detected. This is why recall is kept as the second selection criterion.

Random Forest achieved precision 1.0000 on the test data, meaning no normal traffic was incorrectly marked as anomalous. However, its recall is 0.9985, slightly lower than that of Decision Tree. In a network security context, this difference shows a trade-off between reducing false alarms and capturing more anomalies. Selecting Decision Tree is defensible because the selection rule places F1-score and recall before accuracy.

The model comparison chart helps readers see that performance does not stand alone. If only a table is presented, the difference between models may be difficult to read quickly. Visualization makes the pattern easier to understand, especially when several metrics are compared at once. However, visualization must still be interpreted with narrative explanation because a chart does not explain dataset context, overfitting risk, or benchmark limitations.

The dashboard risk score is derived from the anomaly prediction ratio. In a test dataset with an anomaly ratio close to half of the records, the risk level may appear high. This interpretation is reasonable for a DDoS dataset that intentionally contains many attack records. For production networks, the Low, Medium, and High thresholds must be adjusted to the normal traffic baseline of the organization. In other words, the risk formula in this study is an educational prototype, not a final operational policy.

The local deployment architecture separates the research pipeline from the user interface. Training is conducted in the experiment script, the best model is saved, and the dashboard loads results and prediction files. This separation is important because an operational application should not retrain the model every time a user opens the dashboard. The design also makes future development easier, such as adding authentication, experiment history, and a prediction API.

Academically, the study satisfies the implementation objective because it contains a dataset, code, model, evaluation results, visualizations, and article manuscript. The connection among artifacts makes the research easier to audit. If an examiner wants to check the F1-score, the metric file can be opened. If the examiner wants to inspect prediction output, the CSV file is available. If the examiner wants to read visual interpretation, the images and dashboard provide a consistent summary.

Although the numerical results are very high, the claim of the study remains limited. The model is trained on one DDoS attack type from one benchmark dataset. Attacks such as Botnet, Brute Force, PortScan, Web Attack, and Infiltration are not tested in this final experiment. Therefore, multiclass development and cross-dataset evaluation are important steps before the system is positioned as a more general monitoring tool.

The reproducibility package is an important part of the discussion because machine learning research is often difficult to inspect when only the final manuscript is provided. In this project, the folder structure separates the paper, literature mapping, gap analysis, dataset, source code, model, experiment results, visualization, IEEE draft, presentation, deployment evidence, documentation, and submission readiness materials. This separation helps readers trace the relationship between article claims and technical artifacts.

Numerical consistency is also considered. The values in the abstract, result table, comparison chart, and metrics.json file must refer to the same experiment. Without this consistency, an article may look polished but remain difficult to defend. Therefore, the final document is built by taking numbers from experiment output files instead of typing metrics separately in every section.

Ethical aspects must also be considered if the study is extended to real campus data. Network traffic collection should obtain permission, limit sensitive data, and explain the purpose of processing. Public datasets are suitable for initial learning, but

production data has greater privacy risks. Therefore, the recommended initial deployment is batch prediction on permitted data, not direct packet capture without procedure.

From a usage perspective, the dashboard should be positioned as an investigation support tool. Recommended actions should not automatically execute blocking decisions without human validation. At the prototype stage, recommendation messages should guide inspection, such as reviewing logs, checking traffic spikes, and examining connection sources. Automatic policies can be considered only after the model is tested on broader data and risk thresholds are calibrated.

TABLE III. MODEL COMPARISON RESULTS

Model	Accuracy	Precision	Recall	F1-score
Logistic Regression	0.9920	0.9857	0.9985	0.9921
Decision Tree	0.9992	0.9995	0.9990	0.9992
Random Forest	0.9992	1.0000	0.9985	0.9992

Confusion Matrix - Decision Tree

	Pred Normal	Pred Anomaly
Actual Normal	1986	1
Actual Anomaly	2	1998

Interpretasi: kesalahan klasifikasi sangat rendah pada data uji subset CICIDS2017.

Fig. 3. Confusion matrix of the selected best model.

Risk Score Based on Anomaly Ratio

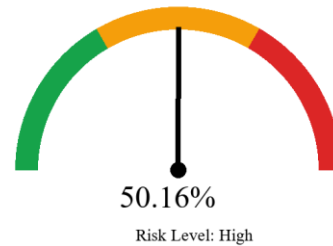


Fig. 4. Risk score visualization based on anomaly ratio.

ROC Curve Illustration

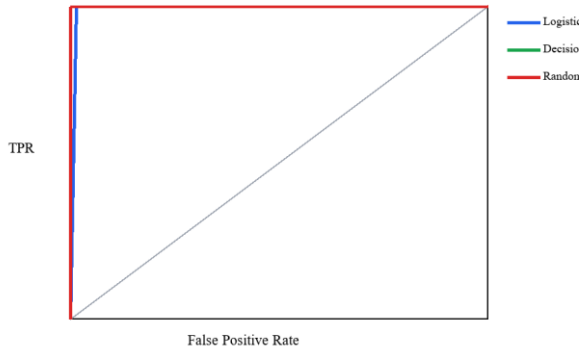


Fig. 5. ROC curve of the selected best model.

Top Feature Association with Label



Catatan: skor ini berbasis korelasi absolut pada subset tes; gunakan feature importance model untuk analisis lanjutan.

Fig. 6. Feature importance summary.

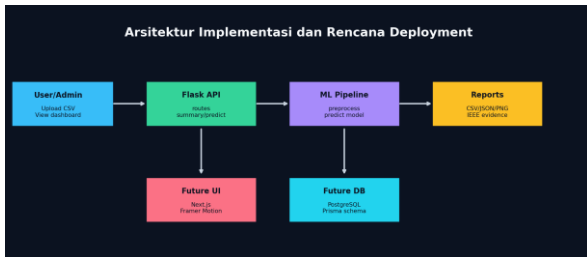


Fig. 7. Deployment architecture and future direction of NetGuard AI.

VI. CONCLUSION

NetGuard AI successfully implements a reproducible machine learning pipeline for network anomaly detection using a CICIDS2017 subset. Among the three tested models, Decision Tree becomes the best model based on an F1-score of 0.9992. The system also provides a lightweight dashboard and a risk classification mechanism that translates model output into more practical monitoring information.

This study shows that the integration of model training, evaluation, visualization, and reproducibility packaging can be built at low cost for educational purposes. Future work should extend the dataset to more attack types, test cross-dataset performance using UNSW-NB15 or CICIDS2018, add model interpretability, improve upload validation, and separate the machine learning service from the frontend if the system is used in an operational environment.

Another important conclusion is that research article quality is not determined only by model performance. Problem clarity, dataset justification, model selection rules, metric interpretation, and reproducibility documentation are equally important. NetGuard AI shows that a student project can be organized into a more mature

IEEE article when implementation, experiment, and discussion are connected to one another.

For future research, evaluation should not only add more data, but also improve scenario variation. Data from several CICIDS2017 days can be combined and compared with CICIDS2018 or UNSW-NB15. The model can also be tested using cross-validation, probability calibration, and more detailed feature importance analysis. If the dashboard is developed for real use, upload security, logging, access control, and traffic collection permission must become part of the design.

Overall, NetGuard AI shows that a simple approach can have research value when it is designed transparently. The main value of the study is not only the high F1-score, but also the way the system connects dataset, model, evaluation, dashboard, and documentation into an inspectable package. With broader validation, this project can develop from a course artifact into a stronger undergraduate thesis foundation or conference paper.

REFERENCES

- [1] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in Proc. ICISPP, 2018, pp. 108-116.
- [2] Z. K. Maseer, R. Yusof, N. Bahaman, S. A. Mostafa, and C. F. M. Foozy, "Benchmarking of machine learning for anomaly based intrusion detection systems in the CICIDS2017 dataset," IEEE Access, vol. 9, pp. 22351-22370, 2021.
- [3] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: The CICIDS2017 case study," in Proc. IEEE Security and Privacy Workshops, 2021, pp. 7-12.
- [4] R. Dube, G. Engelen, V. Rimmer, D. Preuveneers, and W. Joosen, "Faulty use of the CIC-IDS-2017 dataset in information security research," Int. J. Inf. Secur., 2024.
- [5] L. Liu, G. Engelen, T. Lynar, D. Essam, and W. Joosen, "Error prevalence in NIDS datasets: A case study on CIC-IDS-2017 and CSE-CIC-IDS-2018," in Proc. IEEE Conf. Commun. Netw. Secur., 2022.
- [6] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in Proc. IEEE CISDA, 2009, pp. 1-6.
- [7] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems," in Proc. MilCIS, 2015, pp. 1-6.
- [8] M. A. Ferrag, L. Maglaras, S. Moschoviannis, and H. Janicke, "Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study," J. Inf. Secur. Appl., vol. 50, 2020.
- [9] A. Aldweesh, A. Derhab, and A. Z. Emam, "Deep learning approaches for anomaly-based intrusion detection systems: A survey, taxonomy, and open issues," Knowl.-Based Syst., vol. 189, 2020.
- [10] P. Mishra, V. Varadharajan, U. Tupakula, and E. S. Pilli, "A detailed investigation and analysis of using machine learning techniques for intrusion detection," IEEE Commun. Surveys Tuts., vol. 21, no. 1, pp. 686-728, 2019.
- [11] M. Sajid et al., "Enhancing intrusion detection: A hybrid machine and deep learning approach," J. Cloud Comput., vol. 13, no. 123, 2024.
- [12] A. H. Lashkari, G. Draper-Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of Tor traffic using time based features," in Proc. ICISPP, 2017, pp. 253-262.
- [13] L. Breiman, "Random forests," Mach. Learn., vol. 45, no. 1, pp. 5-32, 2001.
- [14] J. R. Quinlan, C4.5: Programs for Machine Learning. San Mateo, CA, USA: Morgan Kaufmann, 1993.
- [15] C. Cortes and V. Vapnik, "Support-vector networks," Mach. Learn., vol. 20, no. 3, pp. 273-297, 1995.
- [16] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825-2830, 2011.
- [17] D. M. W. Powers, "Evaluation: From precision, recall and F-measure to ROC, informedness, markedness and correlation," J. Mach. Learn. Technol., vol. 2, no. 1, pp. 37-63, 2011.

- [18] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," PLoS ONE, vol. 10, no. 3, 2015.
- [19] T. Fawcett, "An introduction to ROC analysis," Pattern Recognit. Lett., vol. 27, no. 8, pp. 861-874, 2006.
- [20] Canadian Institute for Cybersecurity, University of New Brunswick, "CICIDS2017 dataset," accessed Jun. 2026. [Online]. Available: <https://www.unb.ca/cic/datasets/ids-2017.html>
- [21] M. Grinberg, Flask Web Development, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2018.
- [22] Chart.js Contributors, "Chart.js documentation," accessed Jun. 2026. [Online]. Available: <https://www.chartjs.org/docs/latest/>